同濟大學

TONGJI UNIVERSITY

《数据采集与集成技术》

实验报告

实验名称	Flask 框架的网站实现
实验成员	刘彦（2352018）
日　　期	2025 年 10 月 20 日

装 订 线

1、实验目的

本次实验的核心目的在于掌握构建一个基本动态网站的完整流程。通过这个项目，我们旨在打通从后端数据处理到前端页面展示的整个链路。实验不仅仅是学习某一个孤立的技术点，而是要理解服务器（后端）、数据源（文件）和浏览器（前端）三者之间是如何协同工作的，从而对 Web 应用开发建立一个全局性的、实践性的认识。

在技术层面，实验的首要目标是学习并应用 Python 的 Flask 框架。我们通过搭建一个简单的 Web 服务器，学习如何定义路由、处理网络请求，并将后端处理好的数据传递给前端。同时，实验也要求巩固 Python 的文件操作能力，学习如何读取本地文件，并将其中的非结构化文本解析成程序易于处理的结构化数据（如字典列表），这是 Web 开发中常见的数据预处理环节。

2、实验内容

本实验的核心任务是综合运用前后端技术，构建一个完整的动态网站。该网站的最终目标是读取本地文本文件 book.txt 的内容，并将其数据以一个结构化、样式美观且具备客户端交互功能的表格形式，动态地呈现在用户的 Web 浏览器中。

为实现此目标，实验内容涵盖了从服务器端数据处理到客户端页面渲染与交互的完整开发链路，具体划分如下：

2.1 后端开发

后端的职责是作为数据处理中心和 Web 服务器，负责处理原始数据、响应浏览器请求，并为前端提供动态生成的 HTML 内容。

具体来说，首先通过 Python 的 Flask 微型框架搭建一个轻量级 Web 服务器，并配置根路由（/）以监听浏览器的 HTTP GET 请求并执行相应处理逻辑；同时编写代码安全打开并读取 book.txt 文件，采用 UTF-8 编码正确处理中文字符以避免乱码；对文件中每行书籍信息进行解析，转换为结构化的 Python 列表，其中每个元素为字典格式（如{'id': '1', 'title': '书名...', 'category': '分类'}），便于机器处理；最后利用 Flask 集成的 Jinja2 模板引擎，将结构化书籍数据注入预设 HTML 模板，动态生成包含所有书籍信息的完整 HTML 页面，并作为响应返回给用户浏览器，实现高效的数据展示与交互。

2.2 前端开发

前端开发负责接收并渲染后端发送的 HTML 页面，实现所有用户交互的界面展示与功能；利用 HTML 语言构建网页基本骨架，创建语义化的表格（<table>），包含表头（<thead>）和表体（<tbody>），确保前者静态定义，后者能正确接收后端动态填充的书籍数据；通过 CSS 设计清晰美观的整体布局，优化表格可读性（如添加边框、内外边距等），并采用和谐色彩、字体搭配与鼠标悬停高亮交互反馈，提升用户体验；同时运用原生 JavaScript 在浏览器端实现动态交互，包括设置搜索框监听输入事件进行实时客户端过滤、动态显示或隐藏匹配行，以及为表头添加点击监听，根据列数据类型（文本或数字）执行升序/降序切换的表格重新排序，所有操作无需服务器通信，确保高效的本地响应与交互。

3、实验步骤

3.1 项目初始化与环境配置

首先安装 Flask 框架，打开系统的终端（命令行工具），执行以下 pip 命令来安装 Flask。这

是所有后续开发的基础：

```
pip install Flask
```

随后创建项目文件结构，在工作区创建一个项目根目录，在此目录内，手动创建以下文件和文件夹，以构成一个标准的 Flask 项目结构：

```
/root
|-- app.py          # 后端 Flask 应用的主文件
|-- book.txt        # 数据源文件
|-- /templates      # 存放 HTML 模板的文件夹
    |-- index.html # 前端页面文件
```

最后准备数据源 book.txt，确保文件以 UTF-8 编码保存，以避免后续读取时出现中文乱码。

序号	书名	分类
1	HTML5+CSS3+JavaScript 从入门到精通（标准版）	计算机
2	JavaWeb 项目开发实战入门（全彩版）	计算机
3	案例学 WEB 前端开发	计算机
4	一看就停不下来的中国史	历史
5	显微镜下的大明	历史

装

订

线

3.2 后端服务器实现

首先，进行模块导入与初始化，首先从 flask 库中导入 Flask 和 render_template 两个核心组件。然后，通过 app = Flask(__name__)创建 Flask 应用实例。

其次，实现数据读取与解析函数 read_books()，专门负责处理 book.txt 文件。函数逐行读取文件内容。通过 lines[1:]跳过第一行的表头。在循环中，对每一行使用 strip()清除首尾空白，然后用 split()进行分割。考虑到书名中可能包含空格，采用 parts[0]获取序号，parts[-1]获取分类，join(parts[1:-1]) 将中间所有部分合并为书名。接下来进行结构化存储，将解析出的每条书籍信息存为一个 Python 字典，然后将该字典追加到一个名为 books 的列表中，函数最终返回这个列表。

随后，定义路由与视图函数 index()，使用装饰器@app.route('/')将根 URL（/）绑定到 index()函数上。在 index()函数内部，首先调用 read_books()函数获取处理好的书籍数据列表。调用 render_template('index.html', books=book_data)，将数据列表作为名为 books 的变量传递给前端 index.html 模板进行渲染。

最后，启动开发服务器在后端文件的末尾。在该代码块中，调用 app.run(debug=True)来启动 Flask 内置的开发服务器，并开启调试模式，这样在代码修改后服务器会自动重启。

3.3 前端页面构建与交互实现

①构建页面结构（HTML）

页面的主体结构在<body>标签内完成，旨在实现清晰的内容分区和语义化的标签使用：
主容器: 创建一个 作为所有页面内容的中心包装器，便于整体布局和样式控制。在容器内部，首先放置一个<h1>标签用于显示页面主标题“书籍信息展示”。紧接着，创建一个 来包裹搜索框，并通过<input type="text" id="searchInput" ...>创建一个文本输入框，用于用户输入搜索关键词。

创建一个<table id="bookTable">作为展示书籍数据的主体。表格的<thead>部分定义了表头，

包含三个<th>单元格，分别对应“序号”、“书名”和“分类”。为了标识可排序，每个表头单元格内都包含一个 ↕。表格的<tbody>部分是动态内容的载体。它被留空，等待后端通过模板引擎填充。

②动态内容渲染（Jinja2）

为实现数据与视图的分离，我们利用 Flask 的 Jinja2 模板引擎将后端传递的数据动态渲染到 HTML 结构中。在<tbody>标签内部，嵌入{% for book in books %}循环语句。这条语句会遍历后端 app.py 传递过来的 books 列表变量。在循环体内，为列表中的每一项（book）创建一个表格行<tr>，在行内，使用三个<td>单元格，并通过{{ book.id }}, {{ book.title }}, {{ book.category }}表达式，将当前书籍对象的相应属性值精确地填充到单元格中。

③视觉样式设计（CSS）

所有页面的美化工作都在<head>标签内的<style>块中完成，旨在提供一个现代、简洁且用户友好的界面：为 body 设置了基础字体、背景色和文字颜色，构建了页面的基本视觉基调；为.container 类应用了白色背景、圆角边框和 box-shadow，使其呈现为一个悬浮在页面中央的卡片，提升了页面的层次感。

另外，还通过 width: 100%和 border-collapse: collapse 使表格布局规整。为 th 和 td 设置了统一的 padding 和 border-bottom，确保了内容的易读性和行与行之间的清晰分隔。为表头 thead th 设置了醒目的蓝色背景和白色文字，并通过 cursor: pointer 和 user-select: none 明确告知用户其可交互性。利用:hover 伪类为表头和数据行添加了背景色变化的悬停效果，提供了即时的视觉反馈。通过 tbody tr:nth-child(even)实现了“斑马条纹”效果，进一步提高了数据的可读性。。

④客户端交互逻辑（JavaScript）

页面的动态功能完全由浏览器端的 JavaScript 实现，代码位于<body>标签底部的<script>块内，确保在 DOM 加载完毕后执行。

- 实时搜索功能（filterTable 函数）

此函数被绑定到搜索框的 onkeyup 事件上，意味着用户的每一次按键弹起都会触发搜索过滤。函数首先获取用户输入的关键词并转换为大写（以实现不区分大小写的搜索）。然后，它遍历表格的每一数据行（通过 for (let i = 1; ...)跳过表头）。在循环中，它获取该行“书名”和“分类”单元格的文本内容，并检查其中是否包含用户输入的关键词。如果匹配，则设置该行的 style.display 为""（显示）；如果不匹配，则设置为 none（隐藏）。

- 客户端排序功能（sortTable 函数）

此函数被绑定到每个表头<th>的 onclick 事件上，并通过传递列的索引号（0、1 或 2）来指定排序依据。函数内部实现了一个基于 DOM 操作的冒泡排序算法。它使用一个 while 循环，反复遍历表格行，直到没有行需要交换位置为止。通过 dir (direction)和 switchcount（交换计数）变量来智能地处理升序和降序的切换。

函数中最关键的一个细节是，通过 if (columnIndex === 0)判断当前是否在排“序号”列。如果是，则在比较前使用 parseInt() 将从单元格获取的字符串强制转换为整数，从而确保了按数值大小（而非字典顺序）进行正确的排序。当确定两行需要交换位置时，函数直接使用 parentNode.insertBefore()方法来改变它们在文档对象模型（DOM）中的顺序，用户看到的即是排序后的结果。

4、实验结果

在启动后端后，可以看到 book.txt 呈现在网页上。

图 1 网站模型图

还可以进行检索，示例如下：

图 2 网站按书名搜索模型图

图 3 网站按分类搜索模型图

装 订 线

5、实验总结

本次实验通过从零到一构建一个基于 Flask 的动态网页应用，成功地将一个静态的文本文件 book.txt 转换为了一个功能丰富、界面友好的在线书籍展示平台。实验全面覆盖了从后端数据处理到前端界面渲染与交互的全栈开发流程，达到了预期的所有目标。在整个过程中，不仅解决了多个技术难题，也收获了宝贵的开发经验和深刻的体会。

遇到的问题是书名中包含空格时，数据解析发生错位。原因是初期的数据解析逻辑过于简单，直接使用 line.strip().split()方法按任意空白符分割整行文本。这导致了形如《案例学 WEB 前端开发》的书名被错误地分割成多个部分，使得后续的分类信息无法被正确提取。解决方案是调整并优化了解析算法，使其更具鲁棒性。利用数据格式中“序号”总是在最前、“分类”总是在最后的固定规律，改为 parts[0]获取序号，parts[-1]获取分类，然后使用' '.join(parts[1:-1]) 将中间的所有部分重新组合成完整的书名，完美解决了此问题。

本次实验让我真切感受到了 Flask 框架“微”而强大的特性。它没有庞大复杂的结构，上手非常迅速，仅用寥寥数行核心代码便能搭建起一个功能完备的 Web 服务器。同时，其路由定义、模板渲染等核心功能设计得非常直观和精致，让我能够快速聚焦于业务逻辑的实现，而非框架本身的繁琐配置。我还了解了前后端分离思想，这是一个典型的“后端提供数据，前端负责展示”的项目。后端 Python 专注于数据处理，将干净、结构化的数据准备好；而前端 HTML/CSS/JavaScript 则专注于如何将这些数据以最美观、最便捷的方式呈现给用户。特别是将搜索和排序这两个高频交互功能完全放在客户端实现，不仅带来了“零延迟”的流畅体验，也极大地减轻了服务器的计算压力，让我深刻体会到前后端分离在提升应用性能和用户体验上的巨大价值。

附录

附录中是本次实验的代码。

app.py

```
from flask import Flask, render_template
app = Flask(__name__)
def read_books():
    books = []
    try:
        # 确保 'book.txt' 文件与 app.py 在同一目录下
        with open('book.txt', 'r', encoding='utf-8') as f:
            lines = f.readlines()
            # 跳过表头
            for line in lines[1:]:
                parts = line.strip().split()
                if len(parts) >= 3:
                    # 第一个元素是序号，最后一个是分类，中间的是书名
                    book_info = {
                        'id': parts[0],
                        'title': ' '.join(parts[1:-1]),
                        'category': parts[-1]
                    }
                    books.append(book_info)
    except FileNotFoundError:
```

```
        print("错误: 'book.txt' 文件未找到。")
    return books
@app.route('/')
def index():
    book_data = read_books()
    return render_template('index.html', books=book_data)
if __name__ == '__main__':
    app.run(debug=True)
```

templates/index.html

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>书籍列表</title>
    <style>
        body {
            font-family: 'Helvetica Neue', Arial, sans-serif;
            background-color: #f4f7f6;
            color: #333;
            margin: 0;
            padding: 20px;
        }
        .container {
            max-width: 900px;
            margin: 20px auto;
            background-color: #ffffff;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }
        h1 {
            text-align: center;
            color: #2c3e50;
            margin-bottom: 30px;
        }
        .search-container {
            text-align: center;
            margin-bottom: 20px;
        }
        #searchInput {
            width: 50%;
            padding: 12px;
            border: 1px solid #ddd;
            border-radius: 4px;
```

装 订 线

装 订 线

```
            font-size: 16px;
        }
        table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 20px;
        }
        th, td {
            padding: 15px;
            text-align: left;
            border-bottom: 1px solid #ddd;
        }
        thead th {
            background-color: #3498db;
            color: white;
            cursor: pointer;
            user-select: none; /* 防止文字被选中 */
        }
        thead th:hover {
            background-color: #2980b9;
        }
        tbody tr:nth-child(even) {
            background-color: #f9f9f9;
        }
        tbody tr:hover {
            background-color: #f1f1f1;
        }
        .sort-indicator {
            float: right;
            opacity: 0.7;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>书籍信息展示</h1>
        <div class="search-container">
            <input type="text" id="searchInput" onkeyup="filterTable()"
placeholder="按书名或分类搜索...">
        </div>
        <table id="bookTable">
            <thead>
                <tr>
                    <th onclick="sortTable(0)">序号 <span
class="sort-indicator">↕</span></th>
```

```
                    <th onclick="sortTable(1)">书名 <span
class="sort-indicator">↕</span></th>
                    <th onclick="sortTable(2)">分类 <span
class="sort-indicator">↕</span></th>
                </tr>
            </thead>
            <tbody>
                {% for book in books %}
                <tr>
                    <td>{{ book.id }}</td>
                    <td>{{ book.title }}</td>
                    <td>{{ book.category }}</td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
    <script>
        function filterTable() {
            // 获取输入框和表格
            const input = document.getElementById("searchInput");
            const filter = input.value.toUpperCase();
            const table = document.getElementById("bookTable");
            const tr = table.getElementsByTagName("tr");
            // 循环遍历所有表格行，隐藏不匹配的行
            for (let i = 1; i < tr.length; i++) { // 从 1 开始，跳过表头
                const tdTitle = tr[i].getElementsByTagName("td")[1];
                const tdCategory = tr[i].getElementsByTagName("td")[2];
                if (tdTitle || tdCategory) {
                    const titleText = tdTitle.textContent || tdTitle.innerText;
                    const categoryText = tdCategory.textContent ||
tdCategory.innerText;
                    if (titleText.toUpperCase().indexOf(filter) > -1 ||
categoryText.toUpperCase().indexOf(filter) > -1) {
                        tr[i].style.display = "";
                    } else {
                        tr[i].style.display = "none";
                    }
                }
            }
        }
        function sortTable(columnIndex) {
            const table = document.getElementById("bookTable");
            let switching = true;
            // 设置排序方向为升序
            let dir = "asc";
```

```
            let switchcount = 0;
            while (switching) {
                switching = false;
                const rows = table.rows;
                for (let i = 1; i < (rows.length - 1); i++) {
                    let shouldSwitch = false;
                    const x = rows[i].getElementsByTagName("TD")[columnIndex];
                    const y = rows[i +
1].getElementsByTagName("TD")[columnIndex];

                    let xContent = x.innerHTML.toLowerCase();
                    let yContent = y.innerHTML.toLowerCase();
                    // 如果是序号列，则按数字比较
                    if (columnIndex === 0) {
                        xContent = parseInt(xContent);
                        yContent = parseInt(yContent);
                    }
                    if (dir == "asc") {
                        if (xContent > yContent) {
                            shouldSwitch = true;
                            break;
                        }
                    } else if (dir == "desc") {
                        if (xContent < yContent) {
                            shouldSwitch = true;
                            break;
                        }
                    }
                }
                if (shouldSwitch) {
                    rows[i].parentNode.insertBefore(rows[i + 1], rows[i]);
                    switching = true;
                    switchcount++;
                } else {
                    if (switchcount == 0 && dir == "asc") {
                        dir = "desc";
                        switching = true;
                    }
                }
            }
        }
    </script>
</body>
</html>
```